

Search

Reset

TABLE OF CONTENTS

1. GETTING STARTED

2. WHAT YOU WILL NEED

3. AMAZON WEB SERVICES (AWS)

3.1. AWS CLI

3.2. ADMINISTRATOR IAM USER

4. SOLUTION

5. WRITING THE APPLICATION

6. CREATE AN AMAZON S3 BUCKET

7. CONTROLLER API

7.1. UPLOAD ENDPOINT

7.2. DOWNLOAD ENDPOINT

7.3. DELETE ENDPOINT

8. RUNNING THE APPLICATION

9. GENERATE A MICRONAUT APPLICATION NATIVE EXECUTABLE WITH GRAALVM

9.1. GRAALVM INSTALLATION

9.2. NATIVE EXECUTABLE GENERATION

10. TESTING

10.1. UPLOADING A PROFILE PICTURE

10.2. DOWNLOAD A PROFILE PICTURE

10.3. DELETE A PROFILE PICTURE

10.4. CLEANUP

11. NEXT STEPS

12. LICENSE

USE THE MICRONAUT OBJECT STORAGE API TO STORE FILES IN AMAZON S3

Learn how to upload and retrieve files from Amazon S3 using the Micronaut Object Storage API

Authors: Álvaro Sánchez-Mariscal

Micronaut Version: 5.0.0

1. Getting Started

In this guide, we will create a Micronaut application written in Kotlin.

It will be a microservice to store, retrieve and delete profile pictures for users.

2. What you will need

To complete this guide, you will need the following:

- Some time on your hands
- A decent text editor or IDE (e.g. [IntelliJ IDEA](#))
- JDK 21 or greater installed with `JAVA_HOME` [configured appropriately](#).
- An [AWS account](#) with:
 - An IAM user with permissions to create and manage AWS S3 buckets.
 - The AWS CLI configured to use the IAM user above.

3. Amazon Web Services (AWS)

If you don't have one already, [create an AWS Account](#).

3.1. AWS CLI

Follow the [AWS documentation](#) for installing or updating the latest version of the AWS CLI.

3.2. Administrator IAM user

Instead of using your AWS root account, it is recommended that you use an IAM administrative user. If you don't have one already, follow the steps below to create one:

```
aws iam create-group --group-name Administrators
aws iam create-user --user-name Administrator
aws iam add-user-to-group --user-name Administrator --group-name Administrators
aws iam attach-group-policy --group-name Administrators --policy-arn $(aws iam list-policies --query 'Policies[?
PolicyName==`AdministratorAccess`'].{ARN:Arn}') --output text)
aws iam create-access-key --user-name Administrator
```

Copy

Then, run `aws configure` to configure your AWS CLI to use the Administrator IAM user just created.

4. Solution

We recommend that you follow the instructions in the next sections and create the application step by step. However, you can go right to the **completed example**.

- [Download](#) and unzip the source

5. Writing the Application

Create an application using the [Micronaut Command Line Interface](#) or with [Micronaut Launch](#).

```
mn create-app example.micronaut.micronautguide \
  --features=object-storage-aws,graalvm \
  --build=gradle \
  --lang=kotlin \
  --test=junit
```

Copy



If you don't specify the `--build` argument, Gradle with the [Kotlin DSL](#) is used as the build tool.
If you don't specify the `--lang` argument, Java is used as the language.
If you don't specify the `--test` argument, JUnit is used for Java and Kotlin, and Spock is used for Groovy.

The previous command creates a Micronaut application with the default package `example.micronaut` in a directory named `micronautguide`.

If you use Micronaut Launch, select Micronaut Application as application type and add `object-storage-aws`, and `graalvm` features.



If you have an existing Micronaut application and want to add the functionality described here, you can view the dependency and configuration changes from the specified features, and apply those changes to your application.

[Diff](#)

[Setup IntelliJ IDEA to develop Micronaut Applications](#).

6. Create an Amazon S3 bucket

First, we are going to [create an Amazon S3 bucket](#) using the `aws` CLI:

```
aws s3api create-bucket --bucket micronaut-guide-object-storage --region=us-east-1
```



Bucket names must be unique across all AWS accounts.

If you want to create the bucket in a region other than **us-east-1**, you need additional parameters. For example, to create the bucket in the **eu-west-3** region:



```
aws s3api create-bucket \
  --bucket micronaut-guide-object-storage \
  --region eu-west-3 \
  --create-bucket-configuration LocationConstraint=eu-west-3
```

Copy

Then, configure the bucket in **application.properties**:

src/main/resources/application.properties

```
micronaut.object-storage.aws.default.bucket=micronaut-guide-object-storage
```

Copy

7. Controller API

Let’s define an interface with the endpoints of the profile pictures microservice:

src/main/kotlin/example/micronaut/ProfilePicturesApi.kt

```
interface ProfilePicturesApi {

    @Post(uri = "/{userId}", consumes = [MediaType.MULTIPART_FORM_DATA]) ❶
    fun upload(fileUpload: CompletedFileUpload, userId: String, request: HttpRequest<*>): HttpResponse<*>

    @Get("/{userId}") ❷
    fun download(userId: String): Optional<HttpResponse<StreamedFile>>

    @Status(HttpStatus.NO_CONTENT) ❸
    @Delete("/{userId}") ❹
    fun delete(userId: String)
}
```

Copy

- ❶ The [@Post](#) annotation maps the method to an HTTP POST request.
- ❷ The [@Get](#) annotation maps the method to an HTTP GET request.
- ❸ You can return **void** in your controller’s method and specify the HTTP status code via the [@Status](#) annotation.
- ❹ The [@Delete](#) annotation maps the **delete** method to an HTTP Delete request on `/userId`.

And then, an implementation with the required dependencies:

src/main/kotlin/example/micronaut/ProfilePicturesController.kt

```
@Controller(ProfilePicturesController.PREFIX) ❶
@ExecuteOn(TaskExecutors.BLOCKING) ❷
class ProfilePicturesController(
    private val objectStorage: AwsS3Operations, ❸
    private val httpHostResolver: HttpHostResolver ❹
) : ProfilePicturesApi {

    companion object {
        const val PREFIX = "/pictures"
    }
}
```

Copy

- ❶ The class is defined as a controller with the [@Controller](#) annotation mapped to the path `/pictures`.
- ❷ It is critical that any blocking I/O operations (such as fetching the data from the database) are offloaded to a separate thread pool that does not block the Event loop.
- ❸ [AwsS3Operations](#) is the cloud-specific interface for using object storage.
- ❹ [HttpHostResolver](#) allows you to resolve the host for an HTTP

7.1. Upload endpoint

Implement the upload endpoint by receiving the file from the HTTP client via **CompletedFileUpload**, and the **userId** path parameter. Upload it to S3 using [AwsS3Operations](#), and then return its **ETag** in an HTTP response header to the client:

src/main/kotlin/example/micronaut/ProfilePicturesController.kt

```
override fun upload(fileUpload: CompletedFileUpload, userId: String, request: HttpRequest<*>): HttpResponse<*> {
    val key = buildKey(userId) ❶
    val objectStorageUpload = UploadRequest.fromCompletedFileUpload(fileUpload, key) ❷
    val response = objectStorage.upload(objectStorageUpload) { builder -> ❸
        builder.acl(ObjectCannedACL.PUBLIC_READ) ❹
    }

    return HttpResponse
        .created<Any>(location(request, userId)) ❺
        .header(HttpHeaders.ETAG, response.getETag()) ❻
}

private fun buildKey(userId: String): String = "$userId.jpg"

private fun location(request: HttpRequest<*>, userId: String): URI =
    UriBuilder.of(httpHostResolver.resolve(request))
        .path(PREFIX)
        .path(userId)
        .build()
```

- ❶ The key represents the path under which the file will be stored.
- ❷ You can use any of the [UploadRequest](#) static methods to build an upload request.
- ❸ The upload operation returns an [UploadResponse](#), which wraps the cloud-specific SDK response
- ❹ The upload can be customised by using Amazon’s [PutObjectRequest.Builder](#) methods.
- ❺ We return the absolute URL of the resource in the **Location** header.
- ❻ The response object contains some common properties for all cloud vendors, such as the **ETag**, that is sent in a header to the client.

7.2. Download endpoint

For the download endpoint, simply retrieve the entry from the expected key, and transform it into an **StreamedFile**:

src/main/kotlin/example/micronaut/ProfilePicturesController.kt

```
override fun download(userId: String): Optional<HttpResponse<StreamedFile>> {
    val key = buildKey(userId)
    return objectStorage.retrieve(key) ❶
        .map { entry -> buildStreamedFile(entry) } ❷
}

private fun buildStreamedFile(entry: AwsS3ObjectStorageEntry): HttpResponse<StreamedFile> {
    val nativeEntry: GetObjectResponse = entry.nativeEntry
    val mediaType = MediaType.of(nativeEntry.contentType())
    val file = StreamedFile(entry.inputStream, mediaType).attach(entry.key)
    val httpResponse: MutableHttpResponse<Any> = HttpResponse.ok<Any>()
        .header(HttpHeaders.ETAG, nativeEntry.eTag()) ❸
    file.process(httpResponse)
    return httpResponse.body(file)
}
```

- ❶ The retrieve operation returns an **ObjectStorageEntry**, in this case an [AwsS3ObjectStorageEntry](#), which allows accessing the AWS-specific [GetObjectResponse](#) object in case you need more details about the object.
- ❷ We transform the **AwsS3ObjectStorageEntry** into an **HttpResponse<StreamedFile>**.
- ❸ The response contains not only the file, but also an **ETag** header.



The HTTP client could have used the **ETag** from the upload operation and send it in a **If-None-Match** header in the download request [to implement caching](#), which then would have been to be implemented in the download endpoint. But this is beyond the scope of this guide.

7.3. Delete endpoint

For the delete endpoint, all we have to do is invoke the **delete** method with the expected key:

src/main/kotlin/example/micronaut/ProfilePicturesController.kt

```
override fun delete(userId: String) {  
    val key = buildKey(userId)  
    objectStorage.delete(key)  
}
```

8. Running the Application

To run the application, use the `./gradlew run` command, which starts the application on port 8080.

9. Generate a Micronaut Application Native Executable with GraalVM

We will use [GraalVM](#), an advanced JDK with ahead-of-time Native Image compilation, to generate a native executable of this Micronaut application.

Compiling Micronaut applications ahead of time with GraalVM significantly improves startup time and reduces the memory footprint of JVM-based applications.



Only Java and Kotlin projects support using GraalVM’s **native-image** tool. Groovy relies heavily on reflection, which is only partially supported by GraalVM.

9.1. GraalVM Installation

The easiest way to install [GraalVM](#) on Linux or Mac is to use [SDKMan.io](#).

Java 25

```
sdk install java 25.0.2-graal
```

Copy

For installation on Windows, or for a manual installation on Linux or Mac, see the [GraalVM Getting Started](#) documentation.

The previous command installs Oracle GraalVM, which is free to use in production and free to redistribute, at no cost, under the [GraalVM Free Terms and Conditions](#).

Alternatively, you can use the [GraalVM Community Edition](#):

Java 25

```
sdk install java 25.0.2-graalce
```

Copy

9.2. Native Executable Generation

To generate a native executable using Gradle, run:

```
./gradlew nativeCompile
```

Copy

The native executable is created in `build/native/nativeCompile` directory and can be run with `build/native/nativeCompile/micronautguide`.

It is possible to customize the name of the native executable or pass additional parameters to GraalVM:

build.gradle

```
graalvmNative {  
    binaries {  
        main {  
            imageName.set('mn-graalvm-application') ❶  
            buildArgs.add('-Ob') ❷  
        }  
    }  
}
```

Copy

- ❶ The native executable name will now be `mn-graalvm-application`
- ❷ It is possible to pass extra build arguments to **native-image**. For example, `-Ob` enables the quick build mode.

10. Testing

We can test our application from the command line.

10.1. Uploading a profile picture

If you want to upload a file larger than 1MB, you need to configure:



src/main/resources/application.properties

```
# 20 * 1024 * 1024 = 20MB
micronaut.server.multipart.max-file-size=20971520
```

Copy

Assuming you have locally a profile picture in a **profile.jpg** file, you can send it to your application with:

```
$ curl -i -F 'fileUpload=@profile.jpg' http://localhost:8080/pictures/alvaro
```

Copy

```
HTTP/1.1 201 Created
location: http://localhost:8080/pictures/alvaro
ETag: "617cb82e296e153c29b34cccf7af0908"
date: Wed, 14 Sep 2022 12:50:30 GMT
connection: keep-alive
transfer-encoding: chunked
```

Note the **Location** and **ETag** headers.

We can use the **aws** CLI to verify that the file has actually been uploaded to an S3 bucket:

```
aws s3 ls s3://micronaut-guide-object-storage/
```

Copy

10.2. Download a profile picture

```
curl http://localhost:8080/pictures/alvaro -O -J
```

Copy

The file will be saved as **alvaro.jpg** since our download endpoint includes a **Content-Disposition: attachment** header. Open it to check that it is actually the same image as **profile.jpg**.

10.3. Delete a profile picture

```
aws s3 ls s3://micronaut-guide-object-storage/
```

Copy

10.4. Cleanup

Once you are done with this guide, you can remove the bucket from S3 to not leave stale resources:

```
aws s3api delete-bucket --bucket micronaut-guide-object-storage --region eu-west-3
```

Copy

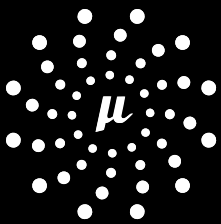
11. Next Steps

- Read more about [Micronaut Object Storage](#).
- Discover [Amazon Simple Storage Service \(Amazon S3\)](#).

12. License



All guides are released with an [Apache license 2.0 license](#) for the code and a [Creative Commons Attribution 4.0](#) license for the writing and media (images...).



M I C R O N A U T
F O U N D A T I O N™

[BRAND GUIDELINES](#)
[COMMUNITY GUIDELINES](#)
[PRIVACY POLICY](#)



GUIDES ARE LICENSED UNDER [CC BY 4.0](#) FOR THE WRITING/MEDIA AND [APACHE 2.0](#) FOR THE CODE